

SRCS : Systèmes Répartis Client/Serveur

TD 9 : Mécanisme de cache réparti

Objectifs pédagogiques

- Interfaces et communication RMI
- Service réparti
- Interaction entre objets RMI

Dans le cadre du TD précédent sur le système de fichier réparti, on se replace dans un monde où les **BlocManager** ne peuvent pas tomber en panne.

On souhaiterait ajouter un mécanisme de cache entre les **BlocManager** et les **FSCClient** pour accélérer les accès sur les données d'un bloc.

Chaque client possède donc un cache local de bloc. Les lectures et écritures se font directement dans le cache et non plus sur l'appel distant au **BlocManager**. Dans un premier temps les écritures sont directement envoyées sur le serveur.

Pour assurer une cohérence forte, le **BlocManager** diffuse chaque mise à jour d'un bloc qu'il héberge à tous les **FSCClient** possédant une copie du bloc.

Question 1

Pourquoi l'objet gérant le cache local doit-il offrir une interface distante ?

Question 2

Modifier l'interface du **BlocManager** pour prendre en compte les accès en cache.

Question 3

Donnez l'interface offerte par le cache ainsi qu'une classe d'implémentation.

Question 4

Quelle hypothèse faut-il faire sur les canaux de communication (justifiez) ?

Question 5

Sachant qu'un bloc est utilisé en moyenne chez M **FSCClient**, quelle est la complexité moyenne (en nombre de messages) de l'algorithme ?

Question 6

Pour réduire la charge des **BlocManager**, lorsqu'un client modifie un bloc, il diffuse directement à tous les autres clients un message invalidant le bloc dans leur cache. La donnée modifiée n'est pas envoyée au **BlocManager**. Quel est l'avantage de cette stratégie par rapport à la précédente ?

Question 7

Décrire un algorithme permettant à un client de charger la dernière version d'un bloc sans effectuer aucune diffusion. Vous définirez clairement les structures de données et variables utiles sur chaque site. Vous considérerez les hypothèses suivantes :

- H1 : les mises à jour ne sont pas transmises au **BlocManager**
- H2 : pour réduire la charge du **BlocManager** celui-ci ne connaît que le premier site client possédant une copie du bloc.

Question 8

De manière générale, quelle est la complexité moyenne en nombre de messages de votre algorithme ?

Question 9

Peut-on réduire le nombre de messages lorsqu'un client réclame à nouveau un bloc qu'il possédait préalablement ? Si oui, modifiez votre algorithme en conséquence.